

MFDP

Multi-Functional Distraction Preventer

Sistem Tasarım Analiz Raporu

07 May 2026

1. Yönetici Özeti

MFDP, Linux masaüstü için geliştirilmiş, gizlilik odaklı, yerel veri tabanlı bir Pomodoro odak takip uygulamasıdır. SQLite üzerine kurulu event-sourcing mimarisiyle granuel oturum olaylarını kaydeder; matplotlib ile zengin analitik görselleştirir. PySide6 (Qt) arayuzu, modern karanlık tema ve sesli geri bildirimle güçlenmiştir.

Proje; mimari kararlar, modülerite ve veri modeli açısından iyi bir temel sunmaktadır. Ancak test altyapısı eksik, bazı dosyalar fazla büyüme ve platform bağımlılığı kapsamlı bir cross-platform kullanımı sınırlandırmaktadır.

2. Amac ve Tasarım Felsefesi

2.1 Amaçlar

- Privacy-first: Kullanıcı verisi tamamen yerel; bulut senkronizasyonu yok.
- Event sourcing: Her oturum sadece özet değil, granuel olaylar (kesinti, milestone, DND değişimi) olarak kaydedilir.
- Modüler mimari: UI, iş mantığı ve veri katmanları birbirinden bağımsız.
- Analitik kapasite: Günlük trend, saatlik iş haritası, odak kalitesi puanı.
- Genişletilebilirlik: Gelecekte ML/pattern detection için event altyapısı hazır.

3. Mimari Analiz

3.1 Katmanlı Mimari

Uygulama üç net katmana ayrılmıştır:

UI Ka

3.2 Kullanılan Tasarım Kalıpları

Kalip	Acıklama
Repository Pattern	Her veri alanı için ayrı repo (SessionRepository, TaskRepository vb.)

Facad

4. Artıları ve Eksileri

4.1 Guclu Yanlar (+)

- + DB katmanı tamamen soyutlanmış; teorik olarak PostgreSQL'e geçiş yapılabilir.
- + Qt Signal/Slot sayesinde UI hiçbir zaman iş mantığını doğrudan çağırılmaz.
- + 5 bağlantılı connection pool ile thread-safe SQLite erişimi sağlanmış.
- + Event sourcing altyapısı: kisintiler, milestone'lar, DND değişimleri kaydedilir.
- + Lazy tab loading: grafik render'i oturum açılışında yapılmaz, bellek dostu.
- + Debounce ile hiyerarşik görev ağacının çoklu sinyal dongusüne karşı önlem alınmış.
- + Soft delete ile kullanıcı verileri hiç kaybolmuyor.
- + Seeder scripti ile analitik testleri için gerçekçi sahte veri üretiliyor.
- + Catppuccin Mocha'dan ilham alan tutarlı ve göz yormayan karanlık tema.
- + Dokümantasyon klasoru (ARCHITECTURE.md, DATABASE_ARCHITECTURE.md) mevcut.

4.2 Zayıf Yanlar / Eksikler (-)

- Test altyapısı tamamen eksik: sıfır unit/integration test mevcut.
- main_window.py 600+ satırla çok büyük; zamanlayıcı kontrolü, sinyal bağlamaları ve UI güncelleme tek sınıfta toplanmış.
- stats_window.py her grafik için ayrı bir metod barındırıyor; ChartRenderer gibi bir yardımcı sınıfa taşınabilir.
- Magic string'ler: 'Focus', 'Short Break', 'Long Break', 'Free Timer' Enum değil; yazım hatası riski taşıyor.
- PomodoroTimer legacy sınıfı hala timer.py içinde; kullanılmıyor, kafa karıştırıyor.
- FocusSession dataclass'ı modeller klasöründe değil, timer.py içinde gömülü.
- Migrasyon sistemi try/except ile yapılmış; versiyon numaralı bir migrasyon sistemi yok.
- DND ve pencere takibi yalnızca Linux/KDE/D-Bus için çalışıyor.
- settings tablosu sadece üç timer süresini tutuyor; tema, dil vb. ayar yok.
- notifier.py içinde ses dosyası yolları hardcoded.
- requirements.txt üretim ve geliştirme bağımlılıkları ayırt etmiyor.

5. Gereksiz / Fazlalık Bileşenler

* PomodoroTimer sınıfı (timer.py)

PmdrCountdownTimer ve CountUpTimer mevcut iken bu legacy sınıf kullanılmıyor. Silinmeli veya yorum satırına alınmalı.

* db_manager.py delegasyon metodları

Birçok metod sadece ilgili repo'yu çağırıyor (tek satır). UI veya core doğrudan repo'ya erişebilir; fazlalık sarma katmanı.

* seeder.py proje kokunde

Gelistirme araci olan seeder, scripts/ veya tools/ altinda olmalı; proje kokunde dagitim paketiyle karisabiliyor.

*** Fazla requirements baglimliligi**

asarPy, fonttools, kiwisolver gibi genellikle otomatik gelen baglimliliklar requirements.txt icinde manuel listelenmis; gereksiz.

6. Esneklik Degerlendirmesi

Seviye	Alan	Aciklama
YUKSEK	Veritabani katmani	BaseRepository sayesinde SQLite'in PostgreSQL ile degistirilmesi teorik olarak mumkun.

7. Muhtemel Gelistirmeler ve Eklemeler

1. Pomodoro Sayaci ve Hedef Sistemi

Kac pomodoro tamamlandigi gosterilmeli ve gunluk/haftalik hedef koymak icin bir arayuz eklenebilir. Bu, cok az mimari degisiklik gerektirir.

2. Veri Disa Aktarimi (Export)

sessions_v2 tablosunun CSV/JSON olarak disari aktarilmasi kullanicilar icin buyuk deger saglar. SessionRepository.export() metodu eklenebilir.

3. Oturum Notlari

sessions_v2 tablosuna 'notes' (TEXT) sutunu eklenerek her oturuma kısa bir not birakilabilir. Migrasyon tek satirlik ALTER TABLE.

4. Haftalik Hedef ve Ilerleme Cubugu

MainWindow'a o haftaki odak dakikasi ve hedefe ulasmak icin gereken kalan sure gosterilebilir. SettingsRepository ile saklanabilir.

5. Enum ile Magic String Temizligi

SessionMode ve EventType icin Python Enum tanimlayarak 'Focus' gibi string literallerini ortadan kaldirmak kod guvenilirligini arttirir.

6. Test Altyapisi

tests/ klasoru olusturulup Repository katmani icin pytest + SQLite in-memory ile unit testler yazilmali. CI/CD icin kritik.

7. Gercek Versiyon Tabanli Migrasyon

migration_version tablosu + sirali migration dosyalari (001_init.sql, 002_add_notes.sql vb.) mevcut try/except yaklasiminin yerine gecilebilir.

8. Cross-Platform DND ve Pencere Takibi

DnDManager ve SystemMonitor icin platform dispatch mantigi: Linux (D-Bus), Windows (winrt), macOS (pyobjc) backend'leri.

9. Atomic Events ile Makine Öğrenmesi

Mevcut altyapı (event_type, elapsed_seconds, metadata) ML için hazır. Scikit-learn ile basit bir odak kalitesi tahmin modeli eklenebilir.

10. Bildirim Entegrasyonu

libnotify veya Qt QSystemTrayIcon ile OS seviyesinde bildirim gönderilebilir; oturum tamamlandığında sesle birlikte görsel bildirim.

11. Tema Seçici

styles.py'daki renkleri bir sozluce tasıyıp SettingsDialog'dan Catppuccin Mocha / Latte / Frappe seçimi yapılabilir.

12. Plugin / Entegrasyon Sistemi

Jira, Toggl, Notion gibi üretkenlik araçlarıyla entegrasyon için hooks veya plugin sistemi eklenebilir.

8. Kod Kalitesi Değerlendirmesi

8.1 Olumlu Yonlar

- + Repository metodlari tek sorumluluk prensibine (SRP) uyuyor.
- + Qt Signal isimleri aciklayici: task_created_signal, finished_signal vb.
- + FocusSession dataclass'i to_db_dict() ile kendini veritabanina donusturuyor.
- + Sinyal dongusunu onlemek icin debounce ve idempotent kontroller mevcut.
- + Sahte veri ureten seeder.py analitik gelistirmeyi kolaylastirir.
- + Baglanti havuzu thread-safe; kuyruk tabanli tasarim dogru.

8.2 iyilestirme Gerektiren Alanlar

! main_window.py cok buyuk

~600 satirla zamanlayici kontrolu, sinyal baglamalari ve UI guncelleme tek sinifta toplanmis. TimerController gibi ayri bir sinifa cikarilmali.

! stats_window.py cok buyuk

Her grafik icin ayri _render_* metodu var. ChartRenderer sinifi ile grafik cizimi soyutlanmali; StatsWindow sadece layout yonetmeli.

! Magic string'ler

'Focus', 'Short Break', 'Long Break', 'Free Timer', 'SESSION_STARTED' gibi literaller Enum olmalı: SessionMode ve EventType Enum tanimlari.

! FocusSession yanlış yerde

data_models.py sadece Task iceriyor; FocusSession timer.py icinde. Tum veri modelleri models/ klasorunde olmalı.

! Hardcoded yollar

notifier.py icinde ses dosyasi yollari sabit string; resources klasoru uzerinden dinamik cozulmeli.

! Dil karisikligi

Sinif ve metod isimleri Ingilizce ama bazi UI metinleri ve yorum satirlari Turkce; standart bir politika belirlenmeli.

9. Dosyalama Duzeni Değerlendirmesi

9.1 Mevcut Yapi

[OK]mfdp_app/core/

Is mantigi burada — dogru. Timer, TaskManager, AtomicAnalyzer, DnDManager, SystemMonitor, Notifier.

[OK]r

9.2 Onerilen Yapi

mfdp_app/

```
└─ core/                (degismez)
└─ db/                  (degismez)
└─ models/              FocusSession buraya tasınmalı
└─ ui/                  (degismez)
└─ resources/           (degismez)
```

Proje koku:

```
└─ scripts/             seeder.py, generate_report.py buraya
└─ tests/               unit/ ve integration/ alt klasorleriyle
└─ docs/                documentation/ klasoru buraya tasınabilir
└─ requirements.txt      (sadece uretim bagımlılıkları)
└─ requirements-dev.txt  (pytest, black, mypy vb.)
```

10. Genel Degerlendirme Puani

Kategori	Puan	Durum
Mimari Tasarım	78/100	İyi
Kod Kalitesi	65/100	Orta
Dosyalama Düzeni	70/100	Orta-İyi
Esneklik / Genisletilebilirlik	72/100	Orta-İyi
Test Coverage	10/100	Kritik Eksik
Dokümantasyon	60/100	Orta
Genel Ortalama	59/100	Orta

Proje, bir side-project olarak güçlü bir mimari altyapıya sahip. Test eksikliği ve bazı büyük dosyalar geliştirme sürecini yavaşlatan en kritik sorunlar. Event-sourcing altyapısı ile ileriki ML/analitik özellikleri için gerçekçi bir potansiyel mevcut.

11. Oncelikli Eylem Plani

P1 - Kritik

- > tests/ klasoru + pytest kurulumu; en azindan repository katmani icin in-memory SQLite testleri.
- > SessionMode ve EventType Enum'larini tanımla; magic string'leri kaldır.
- > FocusSession dataclass'ini models/data_models.py'e taşı.

P2 - Onemli

- > main_window.py'i bölme: TimerController sinifina zamanlayici is mantigini taşı.
- > stats_window.py: ChartRenderer yardimci sinifi olustur.
- > Versiyon tabanlı migrasyon sistemi: migration_version tablosu + sirali SQL.
- > seeder.py'i scripts/ klasorune taşı.

P3 - Gelistirme

- > PomodoroTimer legacy sinifini kaldır (kullanılmıyor).
- > requirements.txt'i requirements.txt + requirements-dev.txt'e bol.
- > notifier.py'deki hardcoded yollari resources klasoru uzerinden coz.
- > settings tablosuna tema/dil tercihlerini ekle.

P4 - Yeni Ozellikler

- > Pomodoro sayaci ve gunluk hedef arayuzu.
- > Oturum notlari (sessions_v2'e notes sutunu).
- > CSV/JSON export fonksiyonu.
- > Cross-platform DND/pencere takibi.

Bu rapor MFDP projesi kaynak kodu analiz edilerek olusturulmustur. Mimari kararlar ve kod kalitesi degerlendirmeleri proje gelistirme sureci esas alinarak yapilmistir.